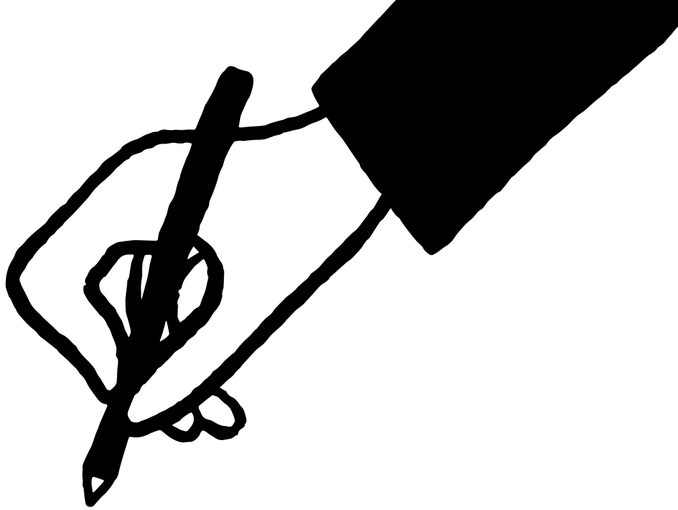


ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

The Composition API

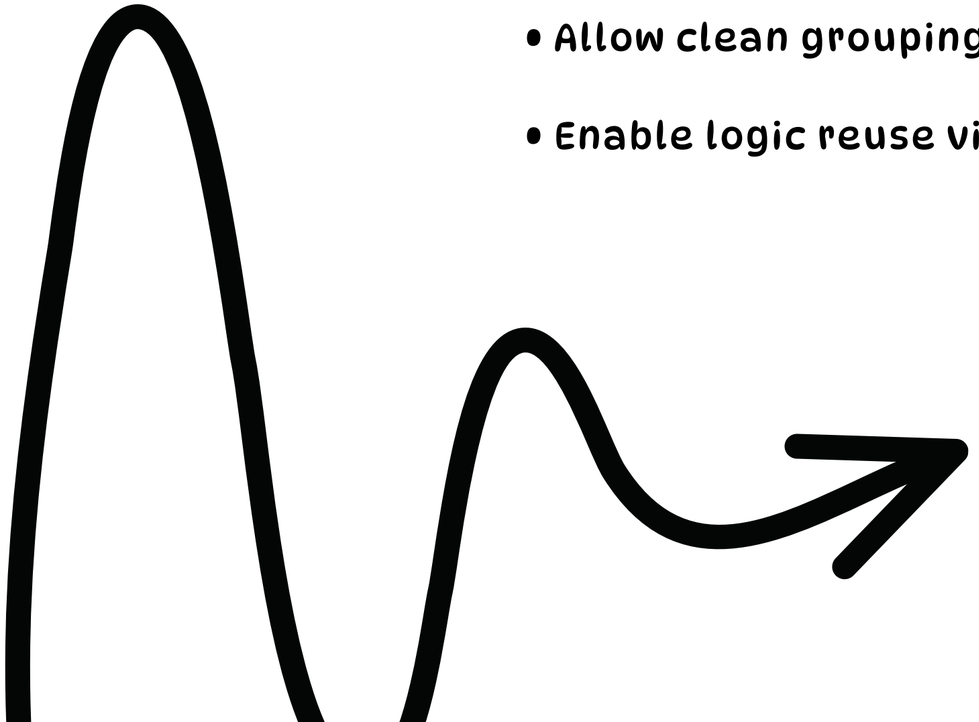
Chapter 5

Advanced Web Design

5 Why Composition API?

Motivation:

- Improve scalability in larger apps
- Provide better TypeScript support
- Allow clean grouping of related logic
- Enable logic reuse via composables



5 Options API vs Composition API Structure

The main difference is how logic is organized:

- Options API organizes by feature type (data, computed, methods, etc.)
- Composition API organizes by logical feature grouping (state, logic, watchers belong together)

Options API

- Good for beginners and small apps
- Logic split across sections

Composition API

- Better for scalable and complex applications
- Logic stays together

Options API vs Composition API

```vue
<template>
  <h2>Count: {{ count }}</h2>
  <button @click="increment">+</button>
</template>

<script>
export default {
  data() {
    return {
      count: 0
    }
  },

  methods: {
    increment() {
      this.count++
    }
  }
}
</script>

```

```vue
<template>
  <h2>Count: {{ count }}</h2>
  <button @click="increment">+</button>
</template>

<script>
import { ref } from 'vue'

export default {
  setup() {
    const count = ref(0)

    function increment() {
      count.value++
    }

    return { count, increment }
  }
}
</script>

```

<script setup> — Even Simpler

Vue provides a shortcut syntax: <script setup> simplifies Composition API syntax.

```vue
<template>
  <h2>Count: {{ count }}</h2>
  <button @click="increment">+</button>
</template>

<script setup>
import { ref } from 'vue'

const count = ref(0)
const increment = () => count.value++
</script>

```

State: data() vs ref() / reactive()

Options API

```vue
<script>
export default {
  data() {
    return {
      name: 'Vue',
      author: 'Marko',
    }
  },
}
</script>
```

Composition API

```vue
<script setup>
import { ref } from 'vue'

const name = ref('Vue')
const author = ref('Marko')
</script>
```

Methods

Options API

```vue
<script>
export default {
  methods: {
    greet() {
      alert(`Hello from ${this.name}`)
    }
  }
}
</script>

```

Composition API

```vue
<script setup>
import { ref } from 'vue'

const name = ref('Vue')

function greet() {
  alert(`Hello from ${name.value}`)
}
</script>

```

Computed Properties

- computed() creates a reactive derived value based on other reactive state.
- It automatically recomputes only when its dependencies change
- It is cached, so as long as the input hasn't changed, it returns the stored value instantly instead of recalculating.

Think of it as a smart value that updates itself.

Computed Properties

Options API

```vue
<script>
export default {
  data() {
    return { count: 2 }
  },
  computed: {
    double() {
      return this.count * 2
    },
  },
}
</script>

```

Composition API

```vue
<script setup>
import { ref, computed } from 'vue'

const count = ref(2)
const double = computed(() => count.value * 2)
</script>
```

Watch

Options API

```vue
<script>
export default {
  data() {
    return { count: 0 }
  },
  watch: {
    count(newVal, oldVal) {
      console.log('Changed', newVal)
    },
  },
}
</script>
```

Composition API

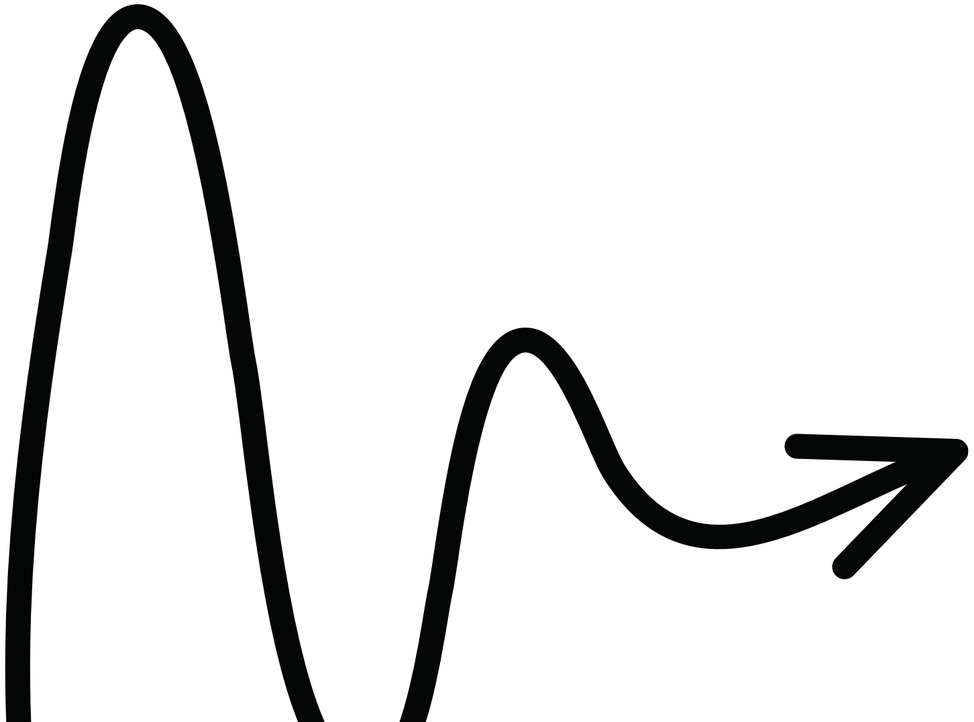
```vue
<script setup>
import { ref, watch } from 'vue'

const count = ref(0)

watch(count, (newVal, oldVal) => {
  console.log('Changed', newVal)
})
</script>
```

watch

- Used when you want to run logic in response to a specific reactive value changing.
- You explicitly tell Vue what value to observe.
- Useful for side effects: API calls, validation, saving to localStorage, logging, etc.
- Not executed immediately by default — only runs when the watched value changes.

watchEffect

- Runs immediately once, and then again every time any reactive value inside it changes.
- Vue automatically tracks dependencies - no need to specify them.
- Great for debugging, syncing UI with model, reactive APIs, or computed-like logic with side effects.
- No access to oldValue (because dependency tracking is automatic).



```
watchEffect(() => {
  console.log('Count changed:', count.value)
  console.log('Name changed:', name.value)
})
```

Full comparison example

```vue
<script>
export default {
  data() {
    return { count: 0 }
  },

  computed: {
    double() {
      return this.count * 2
    },
  },

  watch: {
    count(val) {
      console.log(val)
    },
  },

  methods: {
    increment() {
      this.count++
    },
  },

  mounted() {
    console.log('Mounted')
  },
}
</script>
```

```vue
<script setup>
import { ref, computed, watch, onMounted } from 'vue'

const count = ref(0)
const double = computed(() => count.value * 2)

watch(count, (val) => console.log(val))

function increment() {
  count.value++
}

onMounted(() => console.log('Mounted'))
</script>
```

Lifecycle hooks

Phase	Options API	Composition API
Before mount	beforeMount()	onBeforeMount()
Mounted	mounted()	onMounted()
Updated	updated()	onUpdated()
Unmounted	unmounted()	onUnmounted()

Questions?